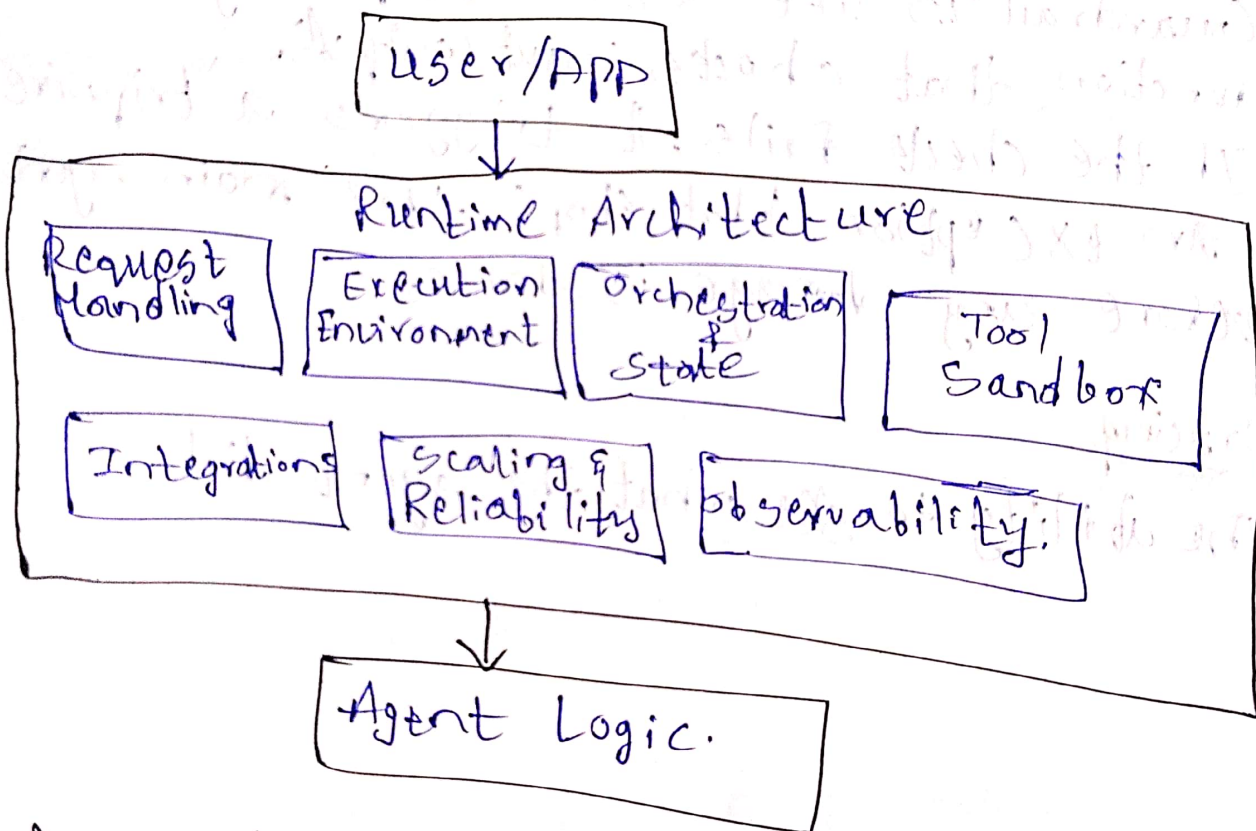


Agentic AI Runtime Architecture

Agent Logic

Runtime Architecture

- * This is like how brain thinks.
 - * The body that lets the brain function.
 - * Multi-layer system that keeps agent logic alive in production.
- Every agent system, from demos to production enterprise, has these three layers.



1) Request Handling

- * This does the authentication, API Gateway routing.

2) Execution Environment

- * Server, container, or serverless ⇒ the machine that runs the agent.

3) Orchestration & State

- * Retries when a tool call fails

- * memory of the process & multi-step workflows.

4) Tool Sandbox

- * Isolation and resource limits when the agent runs code, browses, or hits APIs.

5) Integration layer

- * When the agent reaches the enterprise level then it has to be connected with all the authentication processes such as credentials for databases, API's and enterprise systems.

6) Auto-scaling & reliability

- * To handle the traffic accordingly, load-balancing, failover when traffic spikes or component dies.

7) Observability

- * To see or check step-by-step what the agent did.

Ex: Logs, Traces, & metrics.

⇒ Agent frameworks give you intelligence.

⇒ Runtime architecture lets that intelligence run in the real world.

OCI Enterprise AI platform

1) OCI Enterprise AI Models

- * Model access & managed inference.
- * Can access leading models & run them through a managed service for best price.

2) OCI Enterprise AI Agents

- * Handles Agent management, orchestration, & runtime.

3) OCI Enterprise AI Governance

- * Guardrails, security, & visibility.
 - Network security.
 - Identity Control
 - AI Behavior

1) OCI Enterprise AI Models

a) Chat Models (Conversational AI)

- * Most commonly used types.
- * Handles question answering, summarization, content generation, code, reasoning & agentic workflows.

Ex: Meta Llama 4, Google Gemini 2.5, OpenAI gpt-oss.

b) Emb

* Con
num
me

→ Vect
Com
me

Ex:- C

c) Re

* Th
rel

* Mo
al

Ex:-

OCI

* N
d

Tu

1) B

*

b) Embed Models (Semantic representation).

- * Convert text or images into dense numerical vectors that capture semantic meaning.

→ Vectors are fixed-dimension (eg: 1024-dim), compared via cosine similarity or other measures.

Ex:- Cohere Embed 4.

c) Rerank Models (Relevance Scoring)

- * This enhances the quality of data by relevance scoring.

- * More accurate than vector similarity alone.

Ex:- Cohere Rerank 3.5

OCI Enterprise AI Agents

- * Managed runtime designed for agent development & orchestration.

Two Approaches

1.) Build Agents with OCI Responses API.

- * Run Agentic workloads directly from your application - no infrastructure to deploy.

2) Deploy Hosted Agentic Apps in OCI

- * Package your agent (built with any framework) as a container image, upload to OCI Registry & deploy it as a managed application.

OCI Built-in Tools

1) File Search

- * Semantic retrieval from uploaded documents.

2) Code Interpreter

- * Execute Python in a sandboxed environment.

3) Function Calling

- * Invoking the custom business functions.

4) MCP Calling

- * Connect to remote MCP servers.

Agent Memory

1) Short-Term Memory

- * Remembers the conversation in that particular session.

2) Long-Term Memory

- * Remembers the conversation & context even the session ends.
- * Past decisions, accumulated knowledge etc...

3) Context Optimization

- * Combines the chat history into retained memory.
- * Remembers long conversations for long time.